Module-End -3

Module End Assignment:

Analysing E-Learning Platform Purchases using MySQL

Database Setup & Data Entry

Create a new database.

- Create the three tables with proper:
  - Primary keys, foreign keys, appropriate data types

```sql
-- Create Database
CREATE DATABASE OnlineLearningDB;

-- Use Database
USE OnlineLearningDB;
```

```sql
-- Create Learners Table
CREATE TABLE learners (
    learner_id INT PRIMARY KEY AUTO_INCREMENT,
    full_name VARCHAR(100) NOT NULL,
    country VARCHAR(50) NOT NULL
);
```

```sql
-- Create Courses Table
CREATE TABLE courses (
    course_id INT PRIMARY KEY AUTO_INCREMENT,
    course_name VARCHAR(100) NOT NULL,
    category VARCHAR(50) NOT NULL,
    unit_price DECIMAL(10,2) NOT NULL
);
```

```sql
-- Create Purchases Table
CREATE TABLE purchases (
    purchase_id INT PRIMARY KEY AUTO_INCREMENT,
    learner_id INT NOT NULL,
    course_id INT NOT NULL,
    quantity INT NOT NULL CHECK (quantity > 0),
    purchase_date DATE NOT NULL,

    -- Foreign Keys
    CONSTRAINT fk_learner
        FOREIGN KEY (learner_id)
        REFERENCES learners(learner_id),

    CONSTRAINT fk_course
        FOREIGN KEY (course_id)
        REFERENCES courses(course_id)
);
```

Insert sample data:

- 4–5 learners
- 4–5 courses (different categories)
- 6–8 purchase records

```sql
-- Insert into learners
INSERT INTO learners (full_name, country)
VALUES
('John Smith', 'USA'),
('Priya Sharma', 'India'),
('Ahmed Khan', 'UAE'),
('Maria Garcia', 'Spain'),
('David Lee', 'Singapore');
```

```sql
-- Insert into courses
INSERT INTO courses (course_name, category, unit_price)
VALUES
('SQL for Beginners', 'Database', 199.99),
('Python Programming', 'Programming', 299.99),
('Power BI Dashboarding', 'Data Analytics', 249.99),
('Digital Marketing Basics', 'Marketing', 149.99),
('Advanced Excel', 'Productivity', 179.99);
```

```sql
-- Insert into purchases
INSERT INTO purchases (learner_id, course_id, quantity, purchase_date)
VALUES
(1, 1, 1, '2026-01-10'),
(2, 2, 1, '2026-01-15'),
(3, 3, 2, '2026-02-01'),
(1, 5, 1, '2026-02-10'),
(4, 4, 3, '2026-03-05'),
(5, 2, 1, '2026-03-12'),
(2, 3, 1, '2026-04-08'),
(3, 1, 2, '2026-04-20');
```

2. Data Exploration Using Joins

Use INNER, LEFT, and RIGHT JOIN to:

- Combine learner, course, and purchase data
- *Display:* Learner name, Course name, Category, Quantity, Total amount, Purchase date

INNER JOIN

```sql
SELECT
    l.full_name AS Learner_Name,
    c.course_name AS Course_Name,
    c.category,
    p.quantity,
    (p.quantity * c.unit_price) AS Total_Amount,
    p.purchase_date
FROM purchases p
INNER JOIN learners l
    ON p.learner_id = l.learner_id
INNER JOIN courses c
    ON p.course_id = c.course_id;
```

LEFT JOIN

```sql
SELECT
    l.full_name AS Learner_Name,
    c.course_name AS Course_Name,
    c.category,
    p.quantity,
    (p.quantity * c.unit_price) AS Total_Amount,
    p.purchase_date
FROM learners l
LEFT JOIN purchases p
    ON l.learner_id = p.learner_id
LEFT JOIN courses c
    ON p.course_id = c.course_id;
```

RIGHT JOIN

```sql
SELECT
    l.full_name AS Learner_Name,
    c.course_name AS Course_Name,
    c.category,
    p.quantity,
    (p.quantity * c.unit_price) AS Total_Amount,
    p.purchase_date
FROM purchases p
RIGHT JOIN courses c
    ON p.course_id = c.course_id
LEFT JOIN learners l
    ON p.learner_id = l.learner_id;
```

Presentation Requirements:

- Format currency to 2 decimal places
- Use column aliases
- Sort by the highest total amount

```sql
SELECT
    l.full_name AS Learner_Name,
    c.course_name AS Course_Name,
    c.category AS Category,
    p.quantity AS Quantity,
    FORMAT(p.quantity * c.unit_price, 2) AS Total_Amount,
    p.purchase_date AS Purchase_Date
FROM purchases p
INNER JOIN learners l
    ON p.learner_id = l.learner_id
INNER JOIN courses c
    ON p.course_id = c.course_id
ORDER BY (p.quantity * c.unit_price) DESC;
```

3. Core Analytical Queries (Q1–Q5)

Q1 Display each learner's total spending with their country.

```sql
SELECT
    l.full_name AS Learner_Name,
    l.country AS Country,
    ROUND(SUM(p.quantity * c.unit_price), 2) AS Total_Spending
FROM learners l
INNER JOIN purchases p
    ON l.learner_id = p.learner_id
INNER JOIN courses c
    ON p.course_id = c.course_id
GROUP BY
    l.learner_id,
    l.full_name,
    l.country
ORDER BY
    Total_Spending DESC;
```

Q2. Find the top 3 most purchased courses by quantity.

```sql
SELECT
    c.course_name AS Course_Name,
    c.category AS Category,
    SUM(p.quantity) AS Total_Quantity_Purchased
FROM courses c
INNER JOIN purchases p
    ON c.course_id = p.course_id
GROUP BY
    c.course_id,
    c.course_name,
    c.category
ORDER BY
    Total_Quantity_Purchased DESC
LIMIT 3;
```

Q3. Show each category's:

- Total revenue

- Number of unique learners

```sql
SELECT
    c.category AS Category,
    ROUND(SUM(p.quantity * c.unit_price), 2) AS Total_Revenue,
    COUNT(DISTINCT p.learner_id) AS Unique_Learners
FROM courses c
INNER JOIN purchases p
    ON c.course_id = p.course_id
GROUP BY
    c.category
ORDER BY
    Total_Revenue DESC;
```

Q4. List learners who purchased from more than one category.

```sql
SELECT
    l.full_name AS Learner_Name,
    l.country AS Country,
    COUNT(DISTINCT c.category) AS Categories_Purchased
FROM learners l
INNER JOIN purchases p
    ON l.learner_id = p.learner_id
INNER JOIN courses c
    ON p.course_id = c.course_id
GROUP BY
    l.learner_id,
    l.full_name,
    l.country
HAVING COUNT(DISTINCT c.category) > 1
ORDER BY Categories_Purchased DESC;
```

Q5. Identify courses never purchased.

```sql
SELECT
    c.course_id AS Course_ID,
    c.course_name AS Course_Name,
    c.category AS Category,
    c.unit_price AS Unit_Price
FROM courses c
LEFT JOIN purchases p
    ON c.course_id = p.course_id
WHERE p.purchase_id IS NULL;
```

4. Subqueries & Correlated Subqueries

Q6. Find learners whose total spending is above the average learner spending

```sql
SELECT
    l.learner_id,
    l.full_name AS Learner_Name,
    l.country AS Country,
    ROUND(SUM(p.quantity * c.unit_price), 2) AS Total_Spending
FROM learners l
JOIN purchases p
    ON l.learner_id = p.learner_id
JOIN courses c
    ON p.course_id = c.course_id
GROUP BY
    l.learner_id,
    l.full_name,
    l.country
HAVING SUM(p.quantity * c.unit_price) >
(
    SELECT AVG(total_spending)
    FROM
    (
```

```
        SELECT
            SUM(p.quantity * c.unit_price) AS total_spending
        FROM purchases p
        JOIN courses c
            ON p.course_id = c.course_id
        GROUP BY p.learner_id
    ) AS learner_totals
)
ORDER BY Total_Spending DESC;
```

Q7. Display courses whose price is higher than any course in the 'Beginner' category

```
SELECT
    course_id AS Course_ID,
    course_name AS Course_Name,
    category AS Category,
    unit_price AS Unit_Price
FROM courses
WHERE unit_price >
(
    SELECT MAX(unit_price)
    FROM courses
    WHERE category = 'Beginner'
);
```

Q8 . Find learners who spent more than the average spending in their country.

```sql
SELECT
    lt.Learner_Name,
    lt.Country,
    ROUND(lt.Total_Spending, 2) AS Total_Spending
FROM
(
    SELECT
        l.learner_id,
        l.full_name AS Learner_Name,
        l.country AS Country,
        SUM(p.quantity * c.unit_price) AS Total_Spending
    FROM learners l
    JOIN purchases p
        ON l.learner_id = p.learner_id
    JOIN courses c
        ON p.course_id = c.course_id
    GROUP BY
        l.learner_id,
        l.full_name,
        l.country
```

```sql
) lt
WHERE lt.Total_Spending >
(
    SELECT AVG(country_spending)
    FROM
    (
        SELECT
            l2.country,
            l2.learner_id,
            SUM(p2.quantity * c2.unit_price) AS country_spending
        FROM learners l2
        JOIN purchases p2
            ON l2.learner_id = p2.learner_id
        JOIN courses c2
            ON p2.course_id = c2.course_id
        WHERE l2.country = lt.Country
        GROUP BY l2.country, l2.learner_id
    ) country_avg
)
ORDER BY lt.Country, lt.Total_Spending DESC;
```

5. CTE, CASE, View, and NULL Handling

Q9. Use a CTE to calculate total spending per learner, then:

Display learners with spending above 10,000.

```sql
WITH learner_spending AS (
    SELECT
        l.learner_id,
        l.full_name AS Learner_Name,
        l.country AS Country,
        SUM(p.quantity * c.unit_price) AS Total_Spending
    FROM learners l
    INNER JOIN purchases p
        ON l.learner_id = p.learner_id
    INNER JOIN courses c
        ON p.course_id = c.course_id
    GROUP BY
        l.learner_id,
        l.full_name,
        l.country
)
SELECT
    learner_id AS Learner_ID,
    Learner_Name,
    Country,
```

```sql
)
SELECT
    learner_id AS Learner_ID,
    Learner_Name,
    Country,
    ROUND(Total_Spending, 2) AS Total_Spending
FROM learner_spending
WHERE Total_Spending > 10000
ORDER BY Total_Spending DESC;
```

Q10. CASE Expression

Classify learners based on spending:

- Above 15,000 → “High Value”,
- 8,000–15,000 → “Medium Value”,
- Below 8,000 → “Low Value”.

```
WITH learner_spending AS (
    SELECT
        l.learner_id,
        l.full_name AS Learner_Name,
        l.country AS Country,
        SUM(p.quantity * c.unit_price) AS Total_Spending
    FROM learners l
    INNER JOIN purchases p
        ON l.learner_id = p.learner_id
    INNER JOIN courses c
        ON p.course_id = c.course_id
    GROUP BY
        l.learner_id,
        l.full_name,
        l.countryWITH learner_spending AS (
    SELECT
        l.learner_id,
        l.full_name AS Learner_Name,
        l.country AS Country,
        SUM(p.quantity * c.unit_price) AS Total_Spending
    FROM learners l
    INNER JOIN purchases p
        ON l.learner_id = p.learner_id
    INNER JOIN courses c
        ON p.course_id = c.course_id
    GROUP BY
        l.learner_id,
        l.full_name,
        l.country
)
SELECT
    learner_id AS Learner_ID,
    Learner_Name,
    Country,
    ROUND(Total_Spending, 2) AS Total_Spending,
    CASE
        WHEN Total_Spending > 15000 THEN 'High Value'
        WHEN Total_Spending BETWEEN 8000 AND 15000 THEN 'Medium Value'
```

```
        ELSE 'Low Value'
    END AS Customer_Category
FROM learner_spending
ORDER BY Total_Spending DESC;
```

Q11 . NULL Handling

COALESCE()

```
SELECT
    c.course_id AS Course_ID,
    c.course_name AS Course_Name,
    c.category AS Category,
    COALESCE(SUM(p.quantity), 0) AS Purchase_Count
FROM courses c
LEFT JOIN purchases p
    ON c.course_id = p.course_id
GROUP BY
    c.course_id,
    c.course_name,
    c.category
ORDER BY Purchase_Count DESC;
```

IFNULL()

```sql
SELECT
    c.course_id AS Course_ID,
    c.course_name AS Course_Name,
    c.category AS Category,
    IFNULL(SUM(p.quantity), 0) AS Purchase_Count
FROM courses c
LEFT JOIN purchases p
    ON c.course_id = p.course_id
GROUP BY
    c.course_id,
    c.course_name,
    c.category
ORDER BY Purchase_Count DESC;
```

Q12 . View

- **Create a view: category_performance_view**
- **Showing:**
  - *Category*
  - *Total revenue*
  - *Number of purchases*
  - *Average revenue per purchase*

```sql
CREATE VIEW category_performance_view AS
SELECT
    c.category AS Category,
    ROUND(SUM(p.quantity * c.unit_price), 2) AS Total_Revenue,
    COUNT(p.purchase_id) AS Number_of_Purchases,
    ROUND(
        SUM(p.quantity * c.unit_price) / COUNT(p.purchase_id),
        2
    ) AS Average_Revenue_Per_Purchase
FROM courses c
INNER JOIN purchases p
    ON c.course_id = p.course_id
GROUP BY c.category;
```